Dahlia: A Topological Systems Programming Language

Charlotte W.

July 23, 2026

Abstract

Dahlia is a new systems programming language that uses topological semantics to reason about region-based memory safety mathematically. It models lexical scopes as a Grothendieck site whose sheaf category forms a 1-topos, evaluated as a directed acyclic graph (DAG) during compilation. This approach guarantees zero-cost memory safety without the complexity of an operational borrow checker, while seamlessly enabling mutually recursive data structures and explicit hardware topology mapping.

Contents

1 Introduction

1.1 Motivation

The traditional flat memory model employed by most systems programming languages introduces severe memory safety risks, including dangling pointers, use-after-free errors, and memory leaks. Modern languages like Rust address these issues using a borrow checker; however, this approach introduces significant engineering friction. The borrow checker relies on a complex operational lifetime solver (over 50,000 lines of source code), can be overly restrictive, and makes mutually recursive data structures notoriously difficult to implement, often forcing programmers to fall back on runtime wrappers such as `Box<T>`, `RefCell<T>`, or `Rc<T>`.

Dahlia introduces a fundamentally different paradigm. Rather than treating memory as a flat sequence of bytes governed by temporal lifetime rules, Dahlia models memory spatially. Each scope in the AST is treated as an open set within a Grothendieck site, allowing the compiler's static analysis pass (the "topology checker") to verify memory safety via fast, $O(1)$ reachability checks over a site DAG.

1.2 Thesis

We present Dahlia, a systems language where scope hierarchies and memory arenas are structured as a Grothendieck site whose sheaf category forms a 1-topos. The language is an experiment to explore the practical application of modelling memory safety as a region-based topological problem, rather than a temporal one (lifetime-based).

2 The Dahlia Memory Model

2.1 Spatial Memory and Region Semantics

Dahlia replaces implicit temporal lifetimes with explicit spatial regions denoted by tick annotations (`'s0`, `'s1`). Regions represent distinct bump-pointer arenas tied to AST scopes or custom site topologies.

2.1.1 1. Region Allocation

Lexical scopes and generic parameters define explicit spatial regions. Annotations such as `'s0` explicitly identify objects (open sets) within the underlying site graph.

2.1.2 2. Local Allocation

The built-in operation `local('s0, size)` allocates memory within the bump-pointer arena corresponding to region `'s0`.

2.1.3 3. Topological Reachability Constraints

Generic functions accept explicit morphism paths using directional operators, such as `'s0 ~~> 's1`. This asserts a directed reachability path from region `'s0` to region `'s1`, proving at compile time that data can safely flow between these boundaries.

2.1.4 4. Section Promotion

To transfer data safely out of an inner scope, Dahlia provides the `promote('s0, 's1, ptr)` construct. This operation re-parents an allocation from an inner region `'s1` to an outer region `'s0`, preventing dangling references upon scope termination.

2.2 Language Syntax in Practice

In Dahlia, regions can be nested, and spatial arrows explicitly annotate memory movement:

```
fn alloc_test() void 'r0 {
  var final_str: Str 'r0 = Str.new('r0, "")

  'r1 {
      var inner_str: Str 'r1 = Str.new('r1, "hello")
```

```
    // Explicitly promote string from 'r1 to 'r0
    final_str = promote('r1, 'r0, inner_str) 'r0
}

return final_str 'r0
}

// Function with an explicit directional path constraint between sheaves
fn dependency['r0: sheaf, 'r1: sheaf, A](path 'r0 ~~> 'r1, value: A) A 'r1
return promote('r0, 'r1, value) 'r1
}
```

3 Theoretical Foundations: Sheaves and 1-Topoi

While Dahlia presents a clean region-based abstraction to the programmer (similar to languages like ML-Kit and Cyclone), its static safety proofs are mathematically grounded in sheaf theory over a Grothendieck site, forming a 1-topos.

3.1 Grothendieck Sites and Sheaves

Definition 3.1 (Grothendieck Site)**.** *A* Grothendieck site *is a pair* $(\mathcal{C}, J)$ *where* $\mathcal{C}$ *is a category and* J *is a Grothendieck topology on* $\mathcal{C}$*. The topology* J *assigns to each object* $U \in Ob(\mathcal{C})$ *a collection* $J(U)$ *of covering sieves, which are families of morphisms* $\{U_i \to U\}_{i \in I}$ *satisfying standard intersection, pullback, and transitivity axioms.*

Definition 3.2 (Presheaf)**.** *Let* $\mathcal{C}$ *be a category. A* presheaf *on* $\mathcal{C}$ *is a contravariant functor* $P : \mathcal{C}^{op} \to \mathbf{Set}$*. For any morphism* $f : V \to U$*, the induced map* $P(f) : P(U) \to P(V)$ *is the* restriction map.

Intuition: A presheaf assigns to each spatial region U *a set of valid memory sections (pointers, variables), providing a restriction mechanism to view global data within local sub-scopes.*

Definition 3.3 (Sheaf over a Grothendieck Site)**.** *Let* $(\mathcal{C}, J)$ *be a Grothendieck site. A presheaf* $F : \mathcal{C}^{op} \to \mathbf{Set}$ *is a* sheaf *if for every object* $U \in \mathcal{C}$ *and covering family* $\{f_i : U_i \to U\}_{i \in I} \in J(U)$*:*

1. ***Locality:*** *If* $s, t \in F(U)$ *satisfy* $s|_{U_i} = t|_{U_i}$ *for all* $i \in I$*, then* $s = t$*.*

2. ***Gluing:*** *For any family* $\{s_i \in F(U_i)\}_{i \in I}$ *satisfying* $s_i|_{U_i \times_U U_j} = s_j|_{U_i \times_U U_j}$ *for all* $i, j \in I$*, there exists a unique section* $s \in F(U)$ *such that* $s|_{U_i} = s_i$ *for all* $i \in I$*.*

Intuition: The sheaf axioms guarantee that local allocations can be consistently glued into outer regions. In Dahlia, calling `promote()` *explicitly executes the sheaf gluing axiom to extend a local section across site boundaries.*

3.2 Denotational Semantics

Dahlia treats the scope hierarchy as an open-set category forming a Grothendieck site $\mathcal{C}$. Memory mappings form a sheaf category $\mathrm{Sh}(\mathcal{C}, J)$, which inherently constitutes a Grothendieck 1-topos. During compilation, the topology checker verifies that all memory promotions and restrictions strictly obey the sheaf axioms by lowering site constraints to direct reachability checks on a finite DAG.

4 Formal Type System

This section specifies the core type system used by the language. It is based on Hindley–Milner inference and Algorithm W, presented with bidirectional synthesis and checking judgments. The typing context Γ additionally provides the static information from which the topology checker derives region and reachability constraints described above.

Each rule has one or more premises above the line and a conclusion below it. If every premise holds, then the conclusion is valid.

- Γ is the typing context, mapping names to types.
- τ and τ_r range over types.
- e ranges over expressions, and b ranges over blocks of statements.
- $\Rightarrow$ means a form synthesizes its type from the term itself.
- $\Leftarrow$ means a form is checked against an expected type.
- $\Rightarrow$ is used for statements and declarations that update or preserve the context.
- Items listed as "for each" or written with subscripts such as e_i must hold for every matching element.

4.1 Data Types

- **i8-i64**: A sized integer type (`i8-i64`) with specified bit width. Unsigned integers are denoted with a `u` rather than an `i` prefix.
- **bool**: A boolean type (`bool`) representing true or false values.
- **char**: A single character type (`char`) (8 bits wide).
- **str**: A string type (`str`) representing a sequence of characters.
- **Str**: A boxed string type (`Str`) representing a heap-allocated string.
- **Array**: A fixed-size array type (`[T; N]`) representing a collection of elements of type `T` with size `N`.

4.2 Constants

Constants define values that cannot be changed during program execution. They use the form `const <name>: <type> = <value>`.

4.3 Generic Type Syntax

Polymorphic types are written with square-bracket type parameters, for example `List[A]` or `Result[A, E]`. Type constructors are written in the constructor position, followed by their type arguments in square brackets. A type scheme is written as $\forall \alpha_1, \ldots, \alpha_n. \tau$, where the quantified variables may be instantiated with fresh types during lookup.

$$\textsc{TyApp}\quad \frac{\vdash \tau_1 \text{ type} \qquad \cdots \qquad \vdash \tau_n \text{ type} \qquad C \text{ is a type constructor of arity } n}{\vdash C[\tau_1, \ldots, \tau_n] \text{ type}}$$

$$\textsc{TyScheme}\quad \frac{\vdash \tau \text{ type} \qquad \alpha_1, \ldots, \alpha_n \notin \text{ftv}(\Gamma)}{\Gamma \vdash \forall \alpha_1, \ldots, \alpha_n. \tau \text{ scheme}}$$

This means that `List[A]` is well formed when `List` expects one type argument and `A` is itself a valid type, and similarly for constructors with more parameters such as `Result[A, E]`. A type scheme such as $\forall A.\, List[A] \to A$ is not a distinct runtime type; it is a polymorphic binding that may later be instantiated with fresh type variables.

4.4 Type Checking

The type system employs bidirectional type checking with synthesis and checking. Blocks are typed by the value of their terminal expression or explicit return; an empty block has type `void`.

- **Synthesis**: $\Gamma \vdash e \Rightarrow \tau$ means an expression synthesizes its type from the term in Γ.
- **Checking**: $\Gamma \vdash e \Leftarrow \tau$ means an expression is checked against an expected type τ in Γ.

The rules below are aligned with `GRAMMAR.md` and the current AST. Dahlia does not currently have lambda syntax or explicit type annotations on expressions, so the previous Ann and Abs rules do not apply.

Literal and variable rules

$$\textsc{Lit}\quad \frac{}{\Gamma \vdash \ell \Rightarrow \tau_\ell} \qquad \textsc{Var}\quad \frac{x : \tau \in \Gamma}{\Gamma \vdash x \Rightarrow \tau} \qquad \textsc{ArrayLit}\quad \frac{\Gamma \vdash e_i \Leftarrow \tau \text{ for each } i \in 1..n}{\Gamma \vdash [e_1, \ldots, e_n] \Rightarrow [\tau; n]}$$

Expression rules

CALL
$$\frac{\Gamma \vdash f \Rightarrow (\tau_1, \ldots, \tau_n) \to \tau_r \qquad \Gamma \vdash e_i \Leftarrow \tau_i \text{ for each } i \in 1..n}{\Gamma \vdash f(e_1, \ldots, e_n) \Rightarrow \tau_r}$$

IF
$$\frac{\Gamma \vdash c \Leftarrow \texttt{bool} \qquad \Gamma \vdash b_1 \Rightarrow \tau \qquad \Gamma \vdash b_2 \Rightarrow \tau}{\Gamma \vdash \texttt{if } c\{b_1\} \texttt{ else } \{b_2\} \Rightarrow \tau}$$

ADDR
$$\frac{\Gamma \vdash e \Leftarrow \tau}{\Gamma \vdash \&e \Rightarrow *\tau}$$

DEREF
$$\frac{\Gamma \vdash e \Rightarrow *\tau}{\Gamma \vdash *e \Rightarrow \tau}$$

Operator rules

ARITH
$$\frac{\Gamma \vdash e_1 \Leftarrow \tau \qquad \Gamma \vdash e_2 \Leftarrow \tau \qquad \tau \in \{\texttt{i8}, \texttt{i16}, \texttt{i32}, \texttt{i64}, \texttt{u8}, \texttt{u16}, \texttt{u32}, \texttt{u64}, \texttt{f32}, \texttt{f64}\}}{\Gamma \vdash e_1 \ \oplus \ e_2 \Rightarrow \tau}$$

CMP
$$\frac{\Gamma \vdash e_1 \Leftarrow \tau \qquad \Gamma \vdash e_2 \Leftarrow \tau \qquad \tau \in \{\texttt{i8}, \texttt{i16}, \texttt{i32}, \texttt{i64}, \texttt{u8}, \texttt{u16}, \texttt{u32}, \texttt{u64}, \texttt{f32}, \texttt{f64}\}}{\Gamma \vdash e_1 \ \diamond \ e_2 \Rightarrow \texttt{bool}}$$

EQ
$$\frac{\Gamma \vdash e_1 \Leftarrow \tau \qquad \Gamma \vdash e_2 \Leftarrow \tau}{\Gamma \vdash e_1 \ == \ e_2 \Rightarrow \texttt{bool}}$$

UNARY
$$\frac{\Gamma \vdash e \Leftarrow \tau}{\Gamma \vdash -e \Rightarrow \tau}$$

NOT
$$\frac{\Gamma \vdash e \Leftarrow \texttt{bool}}{\Gamma \vdash !e \Rightarrow \texttt{bool}}$$

Statement rules

VarDecl
$$\frac{\Gamma \vdash e \Leftarrow \tau}{\Gamma \vdash \texttt{var}\ \ x : \tau = e \Rightarrow \Gamma, x : \tau}$$

ConstDecl
$$\frac{\Gamma \vdash e \Leftarrow \tau}{\Gamma \vdash \texttt{const}\ \ x : \tau = e \Rightarrow \Gamma, x : \tau}$$

Assign
$$\frac{x : \tau \in \Gamma \qquad \Gamma \vdash e \Leftarrow \tau}{\Gamma \vdash x = e \Rightarrow \Gamma}$$

While
$$\frac{\Gamma \vdash c \Leftarrow \texttt{bool} \qquad \Gamma \vdash b \Rightarrow \Gamma}{\Gamma \vdash \texttt{while}\ \ c\{b\} \Rightarrow \Gamma}$$

For
$$\frac{\Gamma \vdash it \Rightarrow [\tau; n] \qquad \Gamma, x : \tau \vdash b \Rightarrow \Gamma}{\Gamma \vdash \texttt{for}\ \ x : it\{b\} \Rightarrow \Gamma}$$

Return
$$\frac{\Gamma \vdash e \Leftarrow \tau_r}{\Gamma \vdash \texttt{return}\ \ e \Rightarrow \Gamma}$$

Break
$$\frac{}{\Gamma \vdash \texttt{break} \Rightarrow \Gamma}$$

Declarations and polymorphism

Inst
$$\frac{x : \forall \alpha_1, \ldots, \alpha_n.\, \tau \in \Gamma}{\Gamma \vdash x \Rightarrow \tau[\alpha_1 \mapsto \beta_1, \ldots, \alpha_n \mapsto \beta_n]}$$

Gen
$$\frac{\Gamma \vdash e \Rightarrow \tau \qquad \vec{\alpha} = \mathrm{ftv}(\tau) \setminus \mathrm{ftv}(\Gamma)}{\Gamma \vdash e \rightsquigarrow \forall \vec{\alpha}.\, \tau}$$

VarDecl
$$\frac{\Gamma \vdash e \Leftarrow \tau}{\Gamma \vdash \texttt{var}\ \ x : \tau = e \Rightarrow \Gamma, x : \tau}$$

ConstDecl
$$\frac{\Gamma \vdash e \Rightarrow \tau \qquad \sigma = \mathrm{Gen}(\Gamma, \tau)}{\Gamma \vdash \texttt{const}\ \ x : \tau = e \Rightarrow \Gamma, x : \sigma}$$

FnDecl
$$\frac{\Gamma, x_1 : \tau_1, \ldots, x_n : \tau_n \vdash b \Rightarrow \tau_r \qquad \sigma = \mathrm{Gen}(\Gamma, \tau_1 \to \cdots \to \tau_n \to \tau_r)}{\Gamma \vdash \texttt{fn}\ \ f(x_1 : \tau_1, \ldots, x_n : \tau_n)\{b\} \Rightarrow \Gamma, f : \sigma}$$

- **FnDecl**: function parameters are monomorphic within the body; the function name may be generalized after the body type is known.
- **VarDecl**: mutable bindings remain monomorphic so later assignments preserve a single type.
- **ConstDecl**: immutable bindings may be generalized because their type does not change after binding.

- **StructDecl**: struct fields must have unique names and well-formed types.
- **EnumDecl**: enum variants must have unique names.
- **Generic types**: type constructors use square-bracket syntax such as `List[A]`; the constructor arity must match the number of supplied arguments.
- **Instantiate**: each lookup of a polymorphic binding replaces quantified variables with fresh type variables.
- **Generalize**: when binding a value, quantify only type variables that are free in the inferred type but not free in the environment.
- **Unify**: synthesis and checking meet through unification with occurs-check.